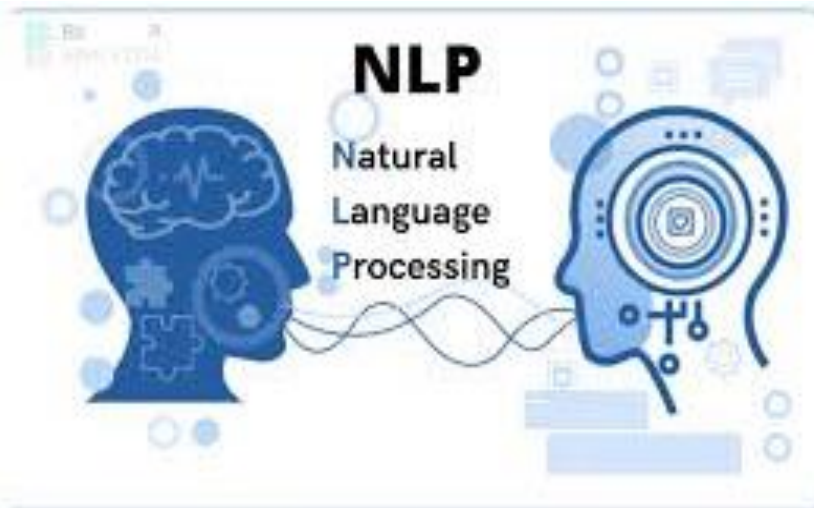




Natural Language Processing

Lecture 1:

Language Processing and Python



1

Dr. Ei Ei Thu
Associate Professor
Faculty of Computer Science

Outlines

What is Natural Language?



Linguistics



Language Processing with Python



Computing with Language

What is Natural Language?

- ❖ In simple terms, a natural language is one developed and evolved by humans through natural use and communication, rather than constructed and created artificially, like a computer programming language.
- ❖ Human languages like English, Japanese, Burmese, and Korean are natural languages.
- ❖ Natural languages can be communicated in different forms, including speech, writing, or even signs.

Linguistics

❖ Formally, linguistics is defined as the scientific study of language, including form and **syntax of language**, meaning, and **semantics** depicted by the usage of language and context of use.



The main distinctive areas of study under linguistics

No	Term	Description
1	Phonetics	Phonetics is the study of the acoustic properties of sounds produced by the human vocal tract during speech. The smallest individual unit of human speech in a specific language is called a phoneme . A more generic term across languages for this unit of speech is phone .

The main distinctive areas of study under linguistics

No	Term	Description
2	Phonology	Phonology is the study of <u>sound patterns</u> as interpreted in the human mind and used for distinguishing between different phonemes to find out which ones are significant. The English language consists of around 44 phonemes (<u>consonant & vowel</u>). Phonology usually extends beyond just studying phonemes and includes things like accents, tone, and syllable structures.

The main distinctive areas of study under linguistics

No	Term	Description
3	Syntax	Syntax is usually the study of sentences, phrases, words, and their structures. It includes researching how words are combined together grammatically to form phrases and sentences. Syntactic order of words used in a phrase or a sentence matter because the order can change the meaning entirely.

The main distinctive areas of study under linguistics

No	Term	Description
4	Semantics	Semantics involves the study of meaning in language and can be further subdivided into lexical and compositional semantics.
	Lexical Semantics	The study of the <u>meanings of words and symbols</u> using morphology and syntax.
	Compositional Semantics	Studying <u>relationships among words</u> and combination of words and understanding the <u>meanings of phrases and sentences</u> and how they are related.

The main distinctive areas of study under linguistics

No	Term	Description
5	Morphology	A morpheme is the <u>smallest unit of language</u> that has <u>distinctive meaning</u> . This includes things like words, prefixes, and suffixes which have their own distinct meanings. Morphology is the <u>study of the structure and meaning</u> of these distinctive units or morphemes in a language. Specific rules and syntaxes usually govern the way morphemes can combine together.

The main distinctive areas of study under linguistics

No	Term	Description
6	Lexicon	Lexicon is the study of <u>properties of words and phrases</u> used in a language and how they build the <u>vocabulary of the language</u>. These include what kinds of sounds are associated with meanings for words, the parts of speech words belong to, and their morphological forms.

The main distinctive areas of study under linguistics

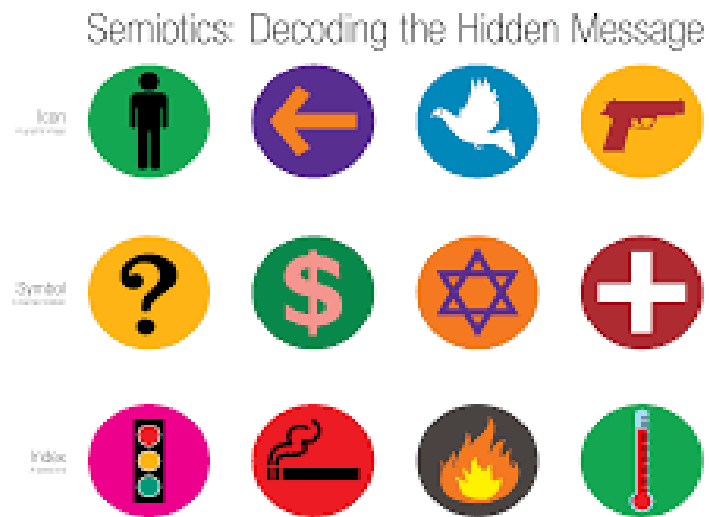
No	Term	Description
7	Pragmatics	Pragmatics is the study of how both linguistic and non-linguistic factors like context and scenario might affect the meaning of an expression of a message or an utterance. This includes trying to infer whether there are any <u>hidden or indirect</u> meanings in the communication.

The main distinctive areas of study under linguistics

No	Term	Description
8	Discourse Analysis	This analyzes language and exchange of information in the form of sentences across <u>conversations</u> among human beings. These conversations could be spoken, written, or even signed.
9	Stylistics	Stylistics is the study of language with a focus on the <u>style of writing</u> , including the tone, accent, dialogue, grammar, and type of voice.

The main distinctive areas of study under linguistics

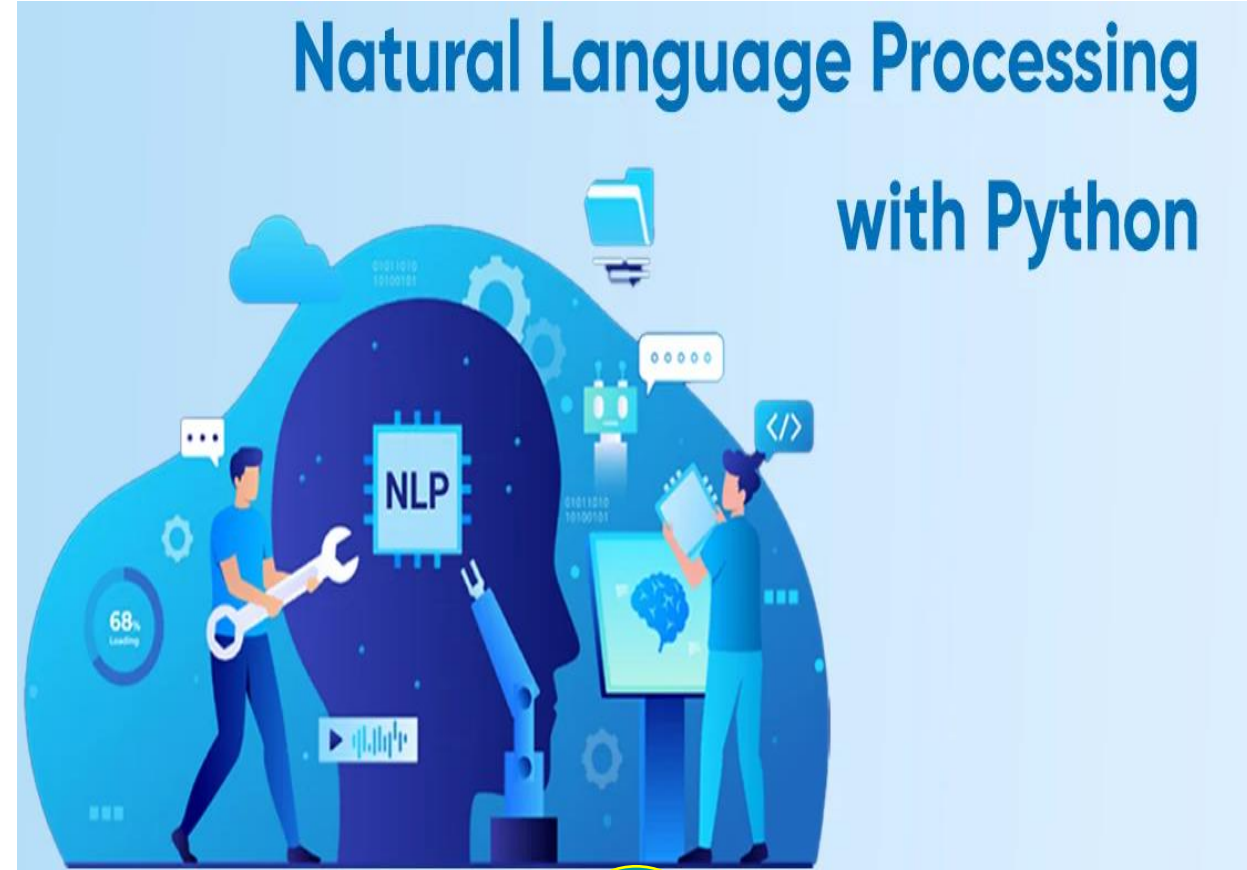
No	Term	Description
10	Semiotics	Semiotics is the study of <u>signs, symbols</u> , and sign processes and how they communicate meaning. Things like analogy, metaphors, and symbolism are covered in this area.



1 Computing with Language:
Texts and Words

2 Computing with Language

3 Automatic Natural Language
Understanding



1

Searching Text

Counting
Vocabulary

Texts as Lists of
Words

2

Simple Statistics

3

Word Sense
Disambiguation

Machine
Translation

Getting Started with NLTK



Select Command Prompt

```
Microsoft Windows [Version 10.0.19044.1288]  
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\dret>pip install nltk_
```

```
pip install nltk
```

Select Command Prompt

```
Microsoft Windows [Version 10.0.19044.1288]  
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\dret>python -m pip install nltk_
```

```
python -m pip install nltk
```

Getting Started with NLTK



```

C:\> Select Command Prompt

Microsoft Windows [Version 10.0.19044.1288]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dret>nltk --version
nltk, version 3.9.2

C:\Users\dret>_
```


Getting Started with NLTK

```
import nltk  
nltk.download('all')
```

 jupyter Eg-1 Last Checkpoint: 12 minutes ago

File Edit View Run Kernel Settings Help

         Code 

[nltk_data] | C:\Users\dret\AppData\Roaming\nltk_data...

```
[ ]: import nltk  
nltk.download('all')
```

What is NLTK?

- ❖ **NLTK (Natural Language Toolkit)** is a comprehensive open-source library in Python designed for working with human language data. It serves as a leading platform for building programs to process and analyze natural language, making it a fundamental tool in the field of Natural Language Processing (NLP). NLTK is widely used by linguists, data scientists, researchers, and students for developing NLP models.

NLTK - NLP



NLTK

NLTK is a versatile library designed to simplify natural language processing with functionalities including text tokenization, tag, and word frequency counting.

01

Tokenize
Text into
Words

02

Segment
Words into
Syllabus

03

Determine
Parts of
Speech

04

Calculate
Word
Frequencies

NLTK - NLP



01



Sentiment analysis
in social media

03



Language learning
and teaching

05



Chatbots and
virtual assistants

Applications of NLTK

02

Information
extraction



04

Machine translation



Text
summarization



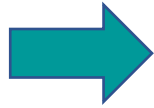
06

Tokenization

- ❖ Tokenization refers to break down the text into smaller units. It splits paragraphs into sentences and sentences into words. It is one of the initial steps of any NLP pipeline. Two major kinds of tokenization that NLTK provides:
- ❖ **Word Tokenization:** it breaks down the text into words.
- ❖ **Sentence Tokenization:** it breaks down the text into individual sentences.

Word Tokenization

Input Text



Ei Ei Thu is an Associate Professor who conducts research primarily in software engineering, focusing on Model-Driven Development (MDD) and the use of object-oriented metrics to measure software quality attributes like modifiability.



```
import nltk
from nltk.tokenize import word_tokenize
```

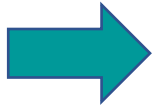
```
text="Ei Ei Thu is an Associate Professor who conducts research primarily in software engineering, focusing on  
tokens=word_tokenize(text)
```

```
print(tokens)
```

```
['Ei', 'Ei', 'Thu', 'is', 'an', 'Associate', 'Professor', 'who', 'conducts', 'research', 'primarily', 'in', 's  
oftware', 'engineering', ',', 'focusing', 'on', 'Model-Driven', 'Development', '(', 'MDD', ')', 'and', 'the',  
'use', 'of', 'object-oriented', 'metrics', 'to', 'measure', 'software', 'quality', 'attributes', 'like', 'modi  
fiability', '.']
```

Sentence Tokenization

**Input
Sentences**



Ei Ei Thu, an associate professor at the University of Computer Studies, Mandalay, has co-authored several academic papers on model-driven development for mobile applications. These papers describe an approach that uses the (Drools knowledge-based rule) engine for transforming design models into object-oriented code.



```
import nltk
from nltk.tokenize import sent_tokenize
```

```
sentences="Ei Ei Thu, an associate professor at the University of Computer Studies, Mandalay, has co-authored s
tokens=sent_tokenize(sentences)
```

```
print(tokens)
```

```
['Ei Ei Thu, an associate professor at the University of Computer Studies, Mandalay, has co-authored several a
cademic papers on model-driven development for mobile applications.', 'These papers describe an approach that
uses the (Drools knowledge-based rule) engine for transforming design models into object-oriented code23.']
```

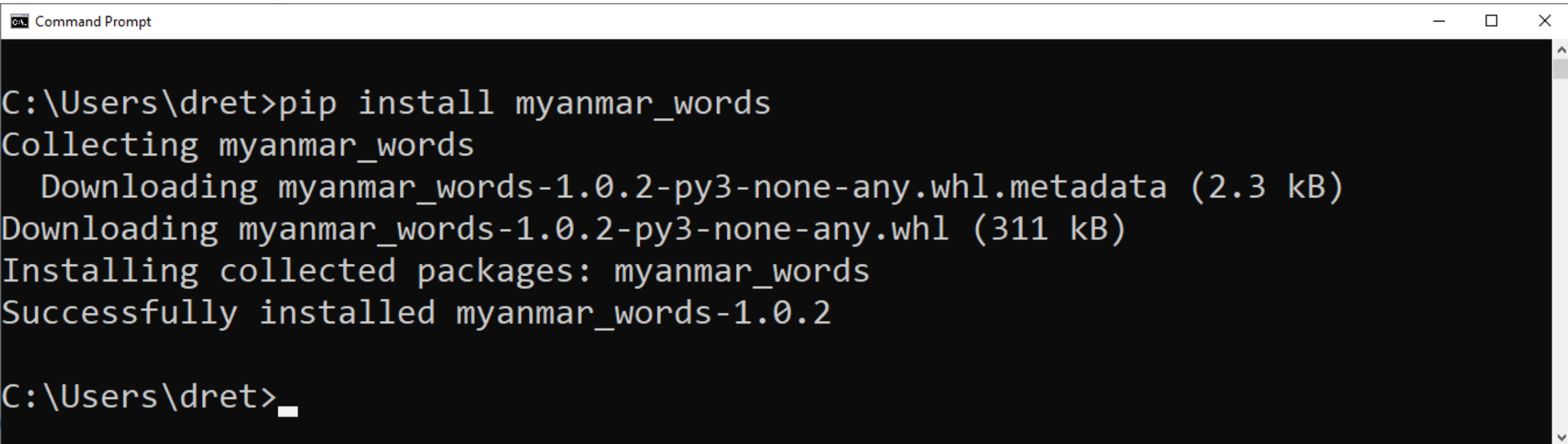

Myanmar Word Tokenizer

myanmar_words is a simple Python library designed for Myanmar (Burmese) text processing, mainly focused on:

- ❖ Word segmentation (word breaking)
- ❖ Word counting
- ❖ Basic word finding (search in text)

It does not provide advanced NLP features like POS tagging or stemming, it is meant to be a lightweight, easy-to-use tokenizer.

Myanmar Word Tokenizer



```
Command Prompt

C:\Users\dret>pip install myanmar_words
Collecting myanmar_words
  Downloading myanmar_words-1.0.2-py3-none-any.whl.metadata (2.3 kB)
Downloading myanmar_words-1.0.2-py3-none-any.whl (311 kB)
Installing collected packages: myanmar_words
Successfully installed myanmar_words-1.0.2

C:\Users\dret>
```

Myanmar Word Tokenizer

```
from myanmar_words import brake, count, find
```

```
text = "မင်္ဂလာပါ ကွန်ပျူတာတက္ကသိုလ် (ပြည်) မှ ကြိုဆိုပါ၏"  
tokens = brake(text)      # word-level token list  
print(tokens)  
print(count(text))        # word count
```

```
['မင်္ဂလာ', 'ပါ', 'ကွန်ပျူတာ', 'တက္ကသိုလ်', '(', 'ပြည်', ')', 'မှ', 'ကြိုဆို', 'ပါ', '၏']  
11
```

Stemming

- ❖ Stemming generates the base word from the given word by removing affixes of the word.
- ❖ It has a set of pre-defined rules that guide the dropping of these affixes.
- ❖ It must be noted that stemmers might not always result in semantically meaningful base words.
- ❖ Stemmers are faster and computationally less expensive than lemmatizers.

Rule-Based (or) Affix-Removed Stemmers

Strip suffixes and prefixes using predefined rules.

- ❖ **Porter Stemmer:** A classic, simple, English-only algorithm, balanced but can over-stem.
- ❖ **Snowball (Porter2):** An improvement on Porter, supports many languages, more accurate, and faster.
- ❖ **Lancaster Stemmer:** Very fast and aggressive, often reduces words to very short forms (over-stemming).
- ❖ **Lovins Stemmer:** Fast, simple, uses longest-suffix removal but less accurate.

How Affix Removal works?

Affix removal is a simple method used in stemming (reducing words to their root form). It works by removing prefixes or suffixes that are attached to a base/root word.

- ❖ Identify common prefixes and suffixes
- ❖ Check if a word starts or ends with those affixes
- ❖ Remove the affix
- ❖ Return the remaining root word

Affix Removal: Simple Example

English

Word: "unhappiness"

Affixes involved:

- Prefix: **un-**
- Suffix: **-ness**

Step-by-step:

1. Check for prefix **un-**
→ "unhappiness" → remove "un" → "**happiness**"
2. Check for suffix **-ness**
→ "happiness" → remove "ness" → "**happy**"

Final root word: **happy**

Myanmar

"မသွားနိုင်တော့ဘူး"

Possible affixes:

- Prefix: **မ-** (negation)
- Suffix: **-နိုင်** (ability)
- Suffix: **-တော့ဘူး** (future negation form)

Step-by-step:

1. Remove prefix **မ-**
→ "သွားနိုင်တော့ဘူး"
2. Remove suffix **-နိုင်**
→ "သွားတော့ဘူး"
3. Remove suffix **-တော့ဘူး**
→ "**သွား**"

Final root word: "**သွား**" ("to go")

Stemming

```
from nltk.stem import PorterStemmer

porter = PorterStemmer()

print(porter.stem("program"))
print(porter.stem("programs"))
print(porter.stem("programming"))
print(porter.stem("programmed"))
print(porter.stem("programmer"))
print(porter.stem("programmable"))
```

program
program
program
program
programm
programm

Lemmatization

Lemmatization is a natural language processing (NLP) technique that reduces words to their base or dictionary form, called a **lemma**, by considering context and meaning, unlike stemming, which just cuts off affixes.

- ❖ Lemmatization uses **vocabulary + grammar rules** to find the correct, meaningful root word.
- ❖ It understands parts of speech (noun, verb, adj,...), word meaning, and grammar.
- ❖ Lemmatizers are slower and computationally more expensive than stemmers.

How Lemmatization works?

- ❖ Identify the word's part of speech (verb?, noun?, adjective?)
- ❖ Check grammar rules (plural?, past tense?)
- ❖ Look up the lemma in a dictionary (not just removing, it uses real vocabulary)
- ❖ Return the correct base form

Stemming vs Lemmatization

Stemming

Cuts affixes mechanically

Fast but inaccurate

"running" → "runn"

"feet" → "fee"

"သွားခဲ့တယ်" → "သွားခဲ့"

Lemmatization

Uses meaning + grammar

Slower but correct

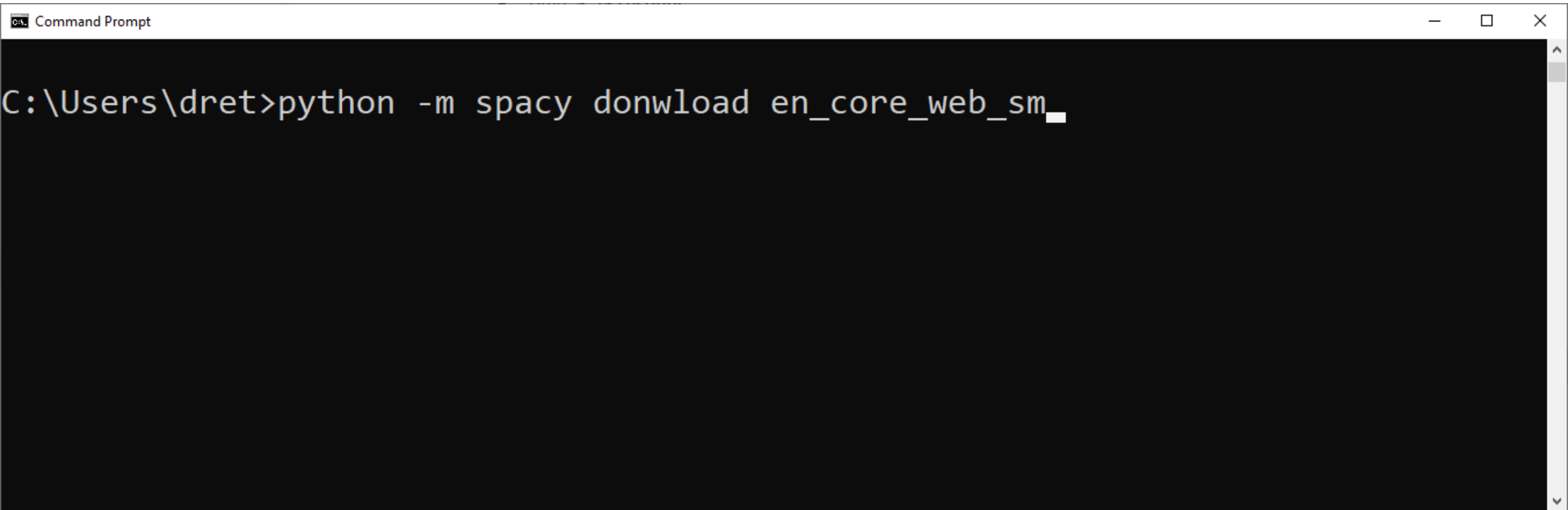
"running" → "run"

"feet" → "foot"

→ "သွား"

```
C:\Users\dret>pip install spacy
Collecting spacy
  Downloading spacy-3.8.11-cp314-cp314-win_amd64.whl.metadata (28 kB)
Collecting spacy-legacy<3.1.0,>=3.0.11 (from spacy)
  Downloading spacy_legacy-3.0.12-py2.py3-none-any.whl.metadata (2.8 kB)
Collecting spacy-loggers<2.0.0,>=1.0.0 (from spacy)
  Downloading spacy_loggers-1.0.5-py3-none-any.whl.metadata (23 kB)
Collecting murmurhash<1.1.0,>=0.28.0 (from spacy)
  Downloading murmurhash-1.0.15-cp314-cp314-win_amd64.whl.metadata (2.3 kB)
Collecting cymem<2.1.0,>=2.0.2 (from spacy)
Successfully installed annotated-types-0.7.0 blis-1.3.3 catalogue-2.0.10 cloudpathlib-0.23.0 confection-0.1.5 cymem-2.0.13 murmurhash-1.0.15 preshed-3.0.12 pydantic-2.12.5 pydantic-core-2.41.5 smart-open-7.5.0 spacy-3.8.11 spacy-legacy-3.0.12 spacy-loggers-1.0.5 srsly-2.5.2 thinc-8.3.10 typer-slim-0.20.0 typing-inspection-0.4.2 wasabi-1.1.3 weasel-0.4.3 wrapt-2.0.1

C:\Users\dret>
```



A screenshot of a Windows Command Prompt window. The title bar at the top reads "C:\> Command Prompt" and includes standard window controls (minimize, maximize, close). The main area of the window is black with white text. The command prompt shows the current directory as "C:\Users\dret" followed by the command "python -m spacy download en_core_web_sm" which is currently being typed, with a white cursor at the end of the line.

```
C:\Users\dret>python -m spacy download en_core_web_sm_
```

Lemmatization: spaCy



spaCy_Test.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help



Share

Q Commands + Code + Text | ▶ Run all ▼

✓ RAM  Disk 



[1]
✓ 12s



```
import spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

```
doc = nlp("The children are playing in the gardens")
```

```
lemmatized = [token.lemma_ for token in doc]  
print(lemmatized)
```

▼ ... ['the', 'child', 'be', 'play', 'in', 'the', 'garden']



Computing with Language: Texts and Words (NLTK)

```
import nltk
from nltk.tokenize import word_tokenize

text = "Natural Language Processing (NLP) is fun. NLP helps in text analysis!"

# 1. Searching text
print("Search 'NLP':", "NLP" in text)

# 2. Text as list of words
words = word_tokenize(text)
print("Text as list of Words:", words)

# 3. Counting vocabulary
unique_words = set(words)
print("Counting Unique Vocabulary:", len(unique_words))
print("Vocabulary List:", unique_words)
```

Computing with Language: Texts and Words (spaCy)



```
import spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

```
text = "Natural Language Processing is fun. NLP helps in text analysis!"
```

```
doc = nlp(text)
```

```
# 1. Searching text
```

```
print("Search 'NLP':", "NLP" in text)
```

```
# 2. Text as list of words
```

```
words = [token.text for token in doc]
```

```
print("Words:", words)
```

```
# 3. Counting vocabulary
```

```
vocab = set(tokens.lemma_ for tokens in doc if tokens.is_alpha)
```

```
print("Unique Vocabulary:", len(vocab))
```

```
print("Vocabulary List:", vocab)
```

```
... Search 'NLP': True
```

```
Words: ['Natural', 'Language', 'Processing', 'is', 'fun', '.', 'NLP', 'helps', 'in', 'text', 'analysis', '!']
```

```
Unique Vocabulary: 10
```

```
Vocabulary List: {'help', 'fun', 'Natural', 'be', 'NLP', 'analysis', 'Language', 'text', 'in', 'processing'}
```

Computing with Language: Simple Statistics – Frequency Distribution

```
import nltk
from nltk import FreqDist
from nltk.tokenize import word_tokenize

text = "Ei Ei Thu, who co-authored papers on developing mobile app frameworks using RESTful web services,"

# Convert text into list of words
words = word_tokenize(text.lower())

# Create frequency distribution
freq = FreqDist(words)

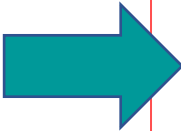
# Show frequencies
print("Frequency of each word:")
for word, count in freq.items():
    print(word, ":", count)

# Most common words
print("\nMost common words:", freq.most_common(5))
```


Computing with Language: Simple Statistics – Frequency Distribution

Input Text

Ei Ei Thu, who co-authored papers on developing mobile app frameworks using RESTful web services, particularly with JSON for data exchange.



Frequency of each word:

```
ei : 2
thu : 1
, : 2
who : 1
co-authored : 1
papers : 1
on : 1
developing : 1
mobile : 1
app : 1
frameworks : 1
using : 1
restful : 1
web : 1
services : 1
particularly : 1
with : 1
json : 1
for : 1
data : 1
exchange : 1
. : 1
```

Most common words: [('ei', 2), (',', 2), ('thu', 1), ('who', 1), ('co-authored', 1)]

Automatic Natural Language Understanding: Word Sense Disambiguation

- ❖ Ambiguity can be referred as the ability of having more than one meaning or being understood in more than one way.

SAME WORD - DIFFERENT MEANINGS



sink



nail



note



fan



button

SAME WORD - DIFFERENT MEANINGS



plant



bark



letter



bat



table

SAME WORD - DIFFERENT MEANINGS



seal



nut



crane



bow



trunk

Automatic Natural Language Understanding: Word Sense Disambiguation

Word Sense Disambiguation (WSD) is the process of determining which meaning of a word is intended in a sentence when the word has multiple meanings. WSD figures out the correct meaning of an ambiguous word based on context. WSD can use:

- ❖ **Rules**
- ❖ Dictionaries
- ❖ Machine learning model
- ❖ Neural network models (BERT, LSTM, etc.)

Word Sense Disambiguation: Example

Word: "bank"

This word has two common meanings:

1. bank = a financial institution
2. bank = the side of a river

Sentence 1:

"John went to the **bank** to deposit money."

→ WSD selects **financial institution**.

Sentence 2:

"John sat on the **bank** and watched the river flow."

→ WSD selects **riverbank**.

```
def wsd_bank(sentence):  
    sentence = sentence.lower()  
  
    river_keywords = ["river", "water", "fish", "shore", "stream"]  
    money_keywords = ["money", "deposit", "loan", "finance", "atm"]  
  
    # Check for river-related context  
    if any(word in sentence for word in river_keywords):  
        return "bank = river bank"  
  
    # Check for financial context  
    if any(word in sentence for word in money_keywords):  
        return "bank = financial institution"  
  
    # Default  
    return "bank = unknown meaning"  
  
# Test sentences  
s1 = "John went to the bank to deposit money."  
s2 = "John sat on the bank and watched the river flow."  
  
print(wsd_bank(s1))  
print(wsd_bank(s2))
```

```
bank = financial institution  
bank = river bank
```

Word Sense Disambiguation: Example

Word: “စာ”

Can mean:

1. စာ = meal
2. စာ = letter (mail)
3. စာ = text / writing

Sentence 1:

“သူ့အတွက် စာ ရေးပြီးပို့လိုက်တယ်။”

→ WSD chooses letter.

Sentence 2:

“ကျောင်းက စာ တွေကို ဖတ်နေတယ်။”

→ WSD chooses text/writing.

Sentence 3:

“ည စာ စားပြီး ပြန်လာတယ်။”

→ WSD chooses meal.

```
def wsd_sa(sentence):
    meal_kw = ["စား", "ည", "နံနက်", "ထမင်း"]
    letter_kw = ["ပို့", "ရေး", "လိပ်စာ", "လွှာ"]
    text_kw = ["ဖတ်", "စာအုပ်", "စာသင်", "စာချုပ်"]

    s = sentence

    if any(k in s for k in meal_kw):
        return "စာ = meal"

    if any(k in s for k in letter_kw):
        return "စာ = letter"

    if any(k in s for k in text_kw):
        return "စာ = writing/text"

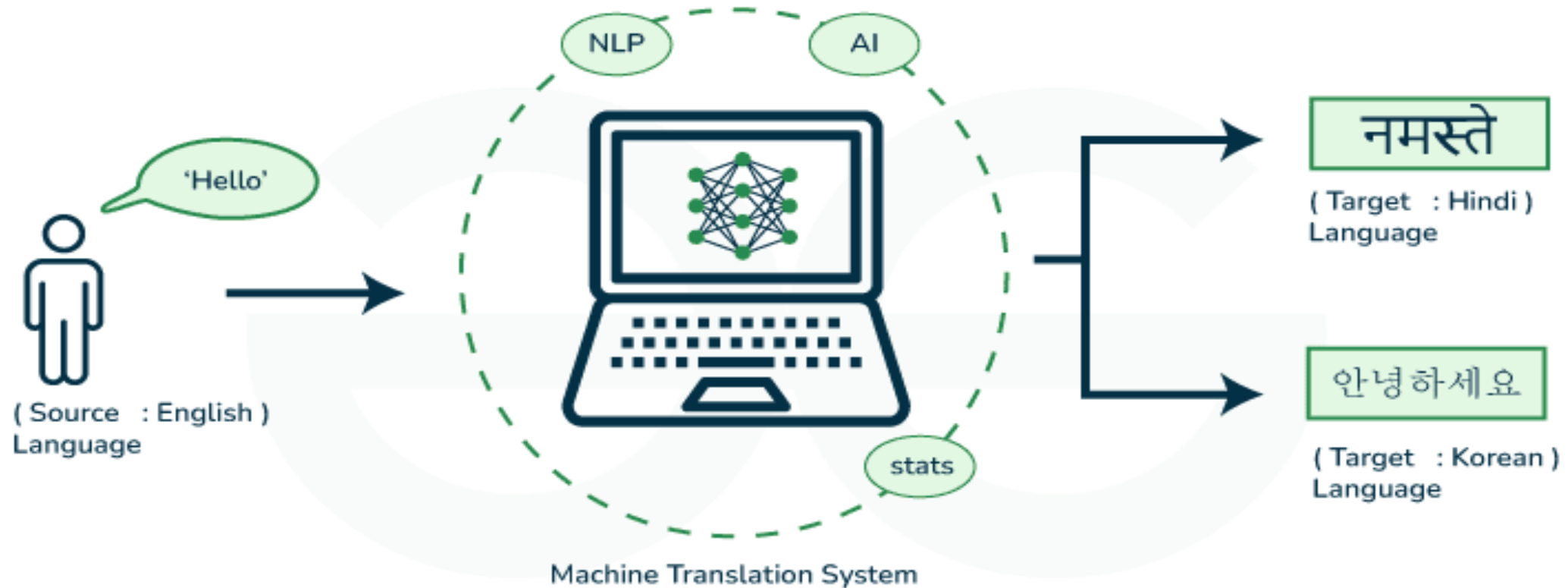
    return "စာ = unknown meaning"

# Test
print(wsd_sa("သူ့ကို စာ ရေးပြီး ပို့လိုက်တယ်။"))
print(wsd_sa("ကျောင်းကစာ တွေကို ဖတ်နေတယ်။"))
print(wsd_sa("ညစာ စားပြီး ပြန်လာတယ်။"))

စာ = letter
စာ = writing/text
စာ = meal
```

Automatic Natural Language Understanding: Machine Translation

Machine Translation in NLP is the task of automatically converting text from one language into another using computational models without human intervention.



Machine Translation: Example

```
dictionary = {  
    "i": "ကျွန်တော်",  
    "you": "သင်",  
    "he": "သူ",  
    "she": "သူမ",  
    "we": "ကျွန်တော်တို့",  
    "they": "သူတို့",  
  
    "eat": "စားတယ်",  
    "go": "သွားတယ်",  
    "reads": "ဖတ်တယ်",  
    "love": "ချစ်တယ်",  
  
    "rice": "ထမင်း",  
    "school": "ကျောင်း",  
    "books": "စာအုပ်"  
}
```

```
def translate_en_to_mm(sentence):  
    words = sentence.lower().split()  
    translated = []  
  
    for w in words:  
        if w in dictionary:  
            translated.append(dictionary[w])  
        else:  
            translated.append(w)  # unknown words kept as-is  
  
    # Simple rule: Myanmar sentences often end with "။"  
    return " ".join(translated) + "။"  
  
# Test examples  
print(translate_en_to_mm("I eat rice"))  
print(translate_en_to_mm("He reads books"))  
print(translate_en_to_mm("We go school"))  
  
ကျွန်တော် စားတယ် ထမင်း။  
သူ ဖတ်တယ် စာအုပ်။  
ကျွန်တော်တို့ သွားတယ် ကျောင်း။
```

References

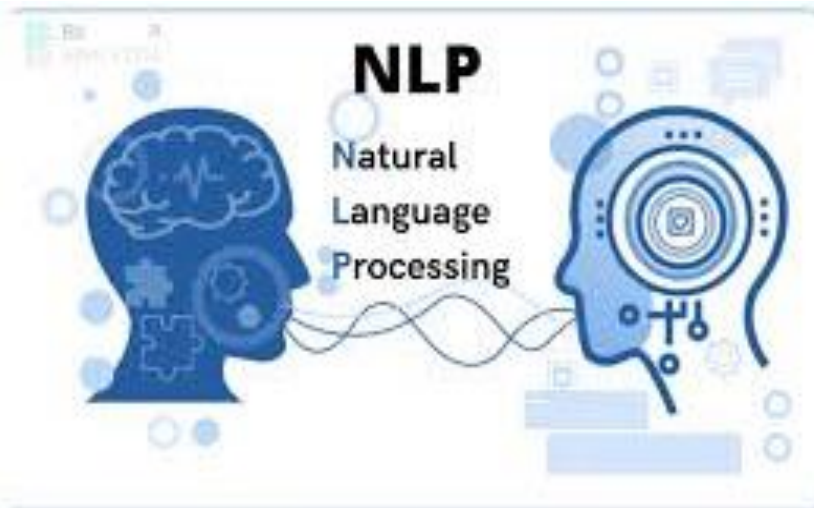
1. Natural Language Processing with Python, Steven Bird, Ewan Klein & Edward Loper, O'Reilly, 2009.
2. Corpus Linguistics, 2nd Edition, Tony McEnery & Andrew Wilson, 2001.
3. Speech and Language Processing_2025, 3rd Draft, Daniel Jurafsky & James H. Martin, 2025.
4. Foundations of Statistical Natural Language Processing, Christopher D. Manning & Hinrich Schutze, 1999.
5. Text Analytics with Python, Dipanjan Sarkar, 2016.



Natural Language Processing

Lecture 2:

Accessing Text Corpora and Lexical Resources



Dr. Ei Ei Thu
Associate Professor
Faculty of Computer Science

Outlines



Accessing
Text Corpora
(NLTK)

Conditional
Frequency
Distributions

What is Text Corpora?

- ❖ A text corpus (Corpora) is a large, structured collection of written or spoken texts, used for linguistic analysis, training AI models, and studying language patterns, word frequencies, syntax, and semantics in a real-world context. A text corpus is a large body of text.

NLTK Text Corpora

- ❖ **Brown Corpus:** The first million-word electronic corpus of English, categorized by genre (news, fiction, romance, etc.).
- ❖ **Gutenberg (selections):** A collection of public domain classic literature texts, such as works by Shakespeare, Jane Austen, and Melville.
- ❖ **Reuters Corpus:** A collection of 10,788 news documents, categorized into 90 overlapping topics (e.g., 'acq', 'barley').

NLTK Text Corpora

- ❖ **Inaugural Address Corpus:** A chronologically organized collection of US Presidential inaugural addresses from 1789 to the present, useful for studying language changes over time.
- ❖ **NPS Chat Corpus:** A collection of instant messaging chat posts, tagged for part-of-speech and dialogue acts.
- ❖ **Movie Reviews:** A corpus of 2,000 movie reviews with sentiment polarity classification (positive/negative).

NLTK Text Corpora

- ❖ **Universal Declaration of Human Rights (UDHR):** Contains the UDHR text in over 300 languages for comparative linguistic studies.
- ❖ **Stopwords Corpus:** A list of common "stopwords" (e.g., 'the', 'is', 'in') for 11 languages, often removed during text preprocessing.
- ❖ **WordNet:** A large lexical database of English, linking words into sets of synonyms (synsets) and defining semantic relationships between them.

NLTK Text Corpora

- ❖ **CMU Pronouncing Dictionary:** Contains pronunciations for over 127,000 English words, useful for text-to-speech applications.
- ❖ **CoNLL and Penn Treebank Corpora:** These corpora feature richer linguistic annotations, such as part-of-speech tags and syntactic parse trees.

LINKING WORDS IN ENGLISH

Emphasis	Addition	Contrast	Order
Undoubtedly Indeed Obviously Particularly / in particular Especially Clearly Importantly Absolutely Definitely Without a doubt Never It should be noted	Additionally / an additional Furthermore Also Too As well as that Along with Besides In addition Moreover Not only...but also In addition to this Apart from this	Unlike Nevertheless On the other hand Nonetheless Despite / in spite of In contrast (to) While Whereas Alternatively Conversely Even so Differing from	First/ firstly Second/ secondly Third/ thirdly Finally At this time Following Previously Before Subsequently Above all Last but not least First and foremost
Result	Illustration	Comparison	Summary
As a result As a consequence (of) Therefore Thus Consequently Hence For this reason Due to	For example/ For instance Such as Including Namely In this case Proof of this like To demonstrate/ To clarify	Similarly Equally Likewise Just as Just like Similar to Same as By the same token In the same way	In conclusion To summarise Altogether In short To sum up In summary Briefly To conclude
Reason	Condition	Concession	Generalisation
Because of With this in mind In fact In order to Due to	If In that case In case Unless	Admittedly All the same Up to a point Even so In spite of Although/Even though Even if However	As a rule For the most part In general/ Generally On the whole Overall In most cases

Source:

<https://7esl.com/wp-content/uploads/2018/05/LINKING-WORDS-IN-ENGLISH11.jpg>

Accessing Text Corpora using Python



Hugging Face

Corpus Access Methods

<https://www.nltk.org/howto/corpus.html>

Method 	Description	Example (using the Brown Corpus)
<code>raw()</code>	The raw content of the corpus as a string.	<code>brown.raw()</code>
<code>words()</code>	A list of words (tokens).	<code>brown.words()</code>
<code>sents()</code>	A list of sentences, where each sentence is a list of words.	<code>brown.sents()</code>
<code>paras()</code>	A list of paragraphs, where each paragraph is a list of sentences.	<code>brown.paras()</code>
<code>fileids()</code>	A list of file identifiers in the corpus.	<code>brown.fileids()</code>
<code>categories()</code>	A list of categories (genres, topics, etc.) if applicable.	<code>brown.categories()</code>

Accessing a Built-in Corpus (NLTK)



```
import nltk
from nltk.corpus import brown

# Download corpus (only once)
nltk.download('brown')

# Access words and sentences
words = brown.words(categories='news')
sentences = brown.sents(categories='news')

print("Number of words:", len(words))
print("First sentence:")
print(' '.join(sentences[0]))
```

`brown.words()` → tokenized words
`brown.sents()` → tokenized sentences

Accessing a Simple Custom Corpus

▶ # Define a small custom corpus

```
english_corpus = [  
    "Natural language processing is interesting.",  
    "Python is widely used in NLP."  
]
```

```
myanmar_corpus = [  
    "သဘာဝဘာသာစကား ပြုလုပ်ခြင်းသည် စိတ်ဝင်စားစရာ ဖြစ်သည်။",  
    "Python ကို NLP တွင် အသုံးများသည်။"  
]
```

Access corpus

```
print("\nEnglish corpus:")  
for s in english_corpus:  
    print(s)
```

```
print("\nMyanmar corpus:")  
for s in myanmar_corpus:  
    print(s)
```

Accessing a Corpus from Hugging Face



```
from datasets import load_dataset

# Load a small English-Myanmar dataset
dataset = load_dataset("kalixlouiis/HFcourse-english-burmese-parallel-corpus")
print(dataset)

print(dataset['train'].column_names)

sample = dataset['train'][0]
print(sample)

print("English:", sample['English'])
print("Myanmar:", sample['Burmese'])
```

Conditional Frequency Distribution

- ❖ **Conditional Frequency Distribution (CFD)** in NLP is a data structure used to **count how often events occur under different conditions.**
- ❖ “frequency counts grouped by a condition.”

Conditional Frequency Distribution



```
import nltk
from nltk import ConditionalFreqDist
from nltk.corpus import brown

nltk.download('brown')

cfd = ConditionalFreqDist(
    (category, word.lower())
    for category in ['news', 'fiction']
    for word in brown.words(categories=category)
)

print("Most common words in news:")
print(cfd['news'].most_common(5))

print("\nMost common words in fiction:")
print(cfd['fiction'].most_common(5))
```

Conditional Frequency Distribution



```
from nltk import ConditionalFreqDist
```

```
corpus = {  
    "English": [  
        "I like NLP",  
        "NLP is interesting"  
    ],  
    "Myanmar": [  
        "NLP ကို ကြိုက်တယ်",  
        "NLP သည် စိတ်ဝင်စားစရာ ဖြစ်သည်"  
    ]  
}
```

```
cfld = ConditionalFreqDist(  
    (lang, word)  
    for lang, sentences in corpus.items()  
    for sent in sentences  
    for word in sent.split()  
)
```

```
print("English word frequency:", cfld['English'].most_common())  
print("Myanmar word frequency:", cfld['Myanmar'].most_common())
```


Exercise

Exercise 1: Accessing an Existing Corpus (NLTK)

Learn how to load and explore a text corpus.

Task:

1. Load the Brown corpus.
2. Print:
 - Number of words
 - Number of sentences
 - First sentence from the *news* category

Exercise 2: Build a Mini Corpus (English + Myanmar)

Task:

Create a small corpus with:

3 English sentences

3 Myanmar sentences

Store them in a dictionary.

Exercise 3: Conditional Frequency Distribution Using Python

Implement CFD in Python.

Task:

Using the corpus from Exercise 2, compute:

Word frequencies **conditioned on language**.

Exercise 4: Hugging Face Corpus Access

Task:

Load any dataset from Hugging Face.

Print:

- Dataset name

- Column names

- First 3 samples

References

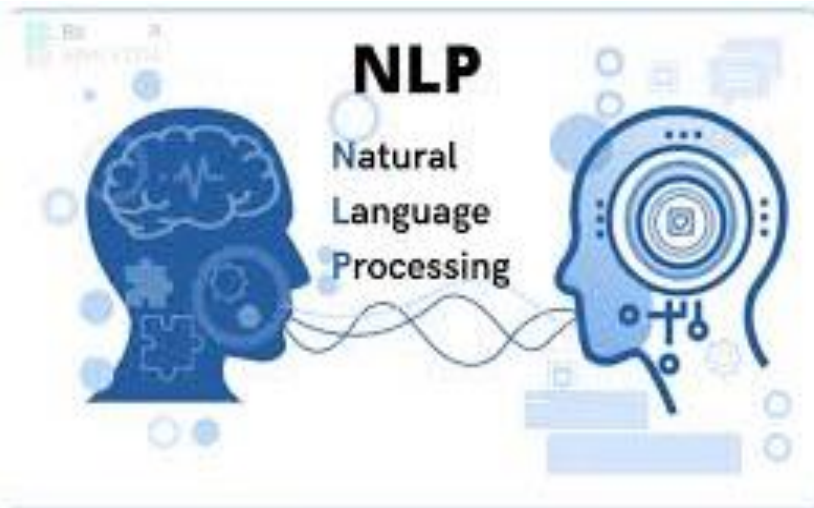
1. Natural Language Processing with Python, Steven Bird, Ewan Klein & Edward Loper, O'Reilly, 2009.
2. Corpus Linguistics, 2nd Edition, Tony McEnery & Andrew Wilson, 2001.
3. Speech and Language Processing_2025, 3rd Draft, Daniel Jurafsky & James H. Martin, 2025.
4. Foundations of Statistical Natural Language Processing, Christopher D. Manning & Hinrich Schutze, 1999.
5. Text Analytics with Python, Dipanjan Sarkar, 2016.
6. <https://www.nltk.org/book/ch02.html>
7. <https://github.com/nltk/nltk>



Natural Language Processing

Lecture 3:

Regular Expressions and Finite State Automata



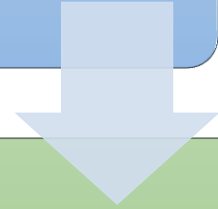
Dr. Ei Ei Thu

Associate Professor

Faculty of Computer Science

Outlines

Regular Expressions



Finite State Automata

Regular Expressions (Regex)

- ❖ Regular expressions (regex or regexp) are a powerful, foundational tool in Natural Language Processing (NLP) used to specify text search patterns for finding, extracting, and manipulating structured or semi-structured information from unstructured text data. They are essential for many text preprocessing and analysis tasks.

Why Regex is Important in NLP?

- ❖ Clean noisy text (URLs, emojis, HTML tags)
- ❖ Tokenization (rule-based)
- ❖ Normalize text (numbers, dates, repeated characters)
- ❖ Extract structured information (emails, phone numbers, IDs)
- ❖ Simple pattern-based classification

```
import re
```

Regular Expressions Function in Python

Function	Description	NLP Use
<code>search()</code>	First match anywhere	Keyword detection
<code>match()</code>	Match at start	Pattern validation
<code>findall()</code>	All matches	Token/entity extraction
<code>finditer()</code>	Match objects	Position tracking
<code>sub()</code>	Replace text	Cleaning
<code>split()</code>	Split text	Sentence/token split
<code>compile()</code>	Reusable regex	Efficiency

`re.search()`

❖ Searches for the **first occurrence** of a pattern anywhere in the string.

```
text="NLP with Python is powerful"  
match=re.search(r'Python', text)  
print(match.group())
```

`r = Raw String`

re.match()

- ❖ Matches a pattern only at the beginning of the string.

```
text="Python for NLP"  
match=re.match(r'Python', text)  
print(match.group())
```

`re.findall()`

❖ Returns **all matches** as a list.

```
text="NLP uses Python and Python libraries"  
matches=re.findall(r'Python', text)  
print(matches)
```

```
re.finditer()
```

❖ Returns an iterator of match objects..

```
text="AI and NLP"  
for match in re.finditer(r'[A-Z]+', text):  
    print(match.group(), match.start(), match.end())
```

re.sub()

❖ Replaces matched patterns with new text.

```
text="NLP 101 course"  
clean=re.sub(r'\d+', "", text)  
print(clean)
```


re.split()

❖ Splits text using a regex pattern.

```
text="NLP is fun. Regex is powerful!"  
sentences=re.split(r'[.!?]', text)  
print(sentences)
```

`re.compile()`

❖ Compiles a regex pattern for reuse and efficiency..

```
pattern=re.compile(r'\b\w+\b')  
tokens=pattern.findall('Regex is useful in NLP')  
print(tokens)
```

Regex Pattern: Basic Symbols

Pattern	Meaning	Example
.	Any character	<code>c.t</code> → cat, cut
^	Start of string	<code>^Hello</code>
\$	End of string	<code>world\$</code>
*	0 or more	<code>lo*</code>
+	1 or more	<code>lo+</code>
?	Optional	<code>colou?r</code>

Regex Pattern: Character Class

Pattern	Meaning
<code>[a-z]</code>	lowercase letters
<code>[A-Z]</code>	uppercase letters
<code>[0-9]</code>	digits
<code>\w</code>	word characters (a-zA-Z0-9_)
<code>\d</code>	digits
<code>\s</code>	whitespace

Character Disjunction: The Square Bracket

Pattern	Match	String
<code>r"[mM]ary"</code>	Mary or mary	" <u>M</u> ary Ann stopped by Mona's"
<code>r"[abc]"</code>	'a', 'b', <i>or</i> 'c'	"In uomini, in sold <u>a</u> ti"
<code>r"[1234567890]"</code>	any one digit	"plenty of <u>7</u> to 5"

Figure 2.9 The use of the brackets `[]` to specify a disjunction of characters.

Character Disjunction: The Square Bracket

Regex	Match	Example Patterns Matched
<code>r"[A-Z]"</code>	an upper case letter	“we should call it ‘ <u>D</u> renched Blossoms’ ”
<code>r"[a-z]"</code>	a lower case letter	“ <u>m</u> y beans were impatient to be hoed!”
<code>r"[0-9]"</code>	a single digit	“Chapter <u>1</u> : Down the Rabbit Hole”

Figure 2.10 The use of the brackets `[]` plus the dash `-` to specify a range.

Character Disjunction: The Square Bracket

Regex	Match (single characters)	Example Patterns Matched
<code>r"[^A-Z]"</code>	not an upper case letter	"O <u>y</u> fn pripetchik"
<code>r"[^Ss]"</code>	neither 'S' nor 's'	" <u>I</u> have no exquisite reason for't"
<code>r"[^.]"</code>	not a period	" <u>o</u> ur resident Djinn"
<code>r"[e^]"</code>	either 'e' or '^'	"look up <u>^</u> now"
<code>r"a^b"</code>	the pattern 'a^b'	"look up <u>a^b</u> now"

Figure 2.11 The caret ^ for negation or just to mean ^. See below re: the backslash for escaping the period.

Counting, Optionality, and Wildcards

Regex	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	zero or one occurrence of the previous char or expression
{n}	exactly <i>n</i> occurrences of the previous char or expression
.	any single char
.*	any string of zero or more chars

Figure 2.12 Counting and wildcards.

Anchors

Regex	Match
^	start of line
\$	end of line
\b	word boundary
\B	non-word boundary

Figure 2.13 Anchors in regular expressions.

Disjunction, Grouping, and Precedence

Parenthesis	()
Counters	* + ? {}
Sequences and anchors	the ^my end\$
Disjunction	

EXAMPLE

Regex for Tokenization

```
text="Regex is useful in NLP!"  
tokens=re.findall(r'\b\w+\b', text)  
print(tokens)
```

Text Cleaning with Regex: Remove Punctuation

```
text="Hello, NLP world!"  
clean=re.sub(r'[^\\w\\s]', '', text)  
print(clean)
```

Text Cleaning with Regex: Remove Numbers

```
re.sub(r'\\d+', '', "NLP 101 course")
```

Text Normalization

```
re.sub(r'\\s+', ' ', "This is NLP")
```

Information Extraction: Extract Email Address

```
text="Contact us at info@example.com"  
emails=re.findall(r'[\w.-]+\@[\w.-]+\.\w+', text)  
print(emails)
```

Information Extraction: Extract Phone Number

```
re.findall(r'\b\d{11}\b', "Call 09789194113")
```

Remove URLs from Text

```
text="Visit https://example.com for more info"  
clean=re.sub(r'http\S+', "", text)  
print(clean)
```

EXERCISE

1. Remove all digits from the text.

“I scored 95 marks in NLP 2025”

2. Extract all hashtags from the text.

“I love #NLP and #MachineLearning”

3. Tokenize the sentence using regex (ignore punctuation).

“Regex-based NLP is powerful!”

4. Remove all punctuation from the sentence.

"Hello!!! Welcome to NLP, 2026."

5. Extract all numbers from the text.

"Batch 2023 has 45 students and 3 instructors"

6. Normalize spaces to a single space.

"This is an NLP class"

7. Extract User Mentions

"Thanks @alice and @bob123"

8. Extract dates from text.

"The exam is on 12-03-2026"

9. Split text into sentences.

"NLP is fun. Regex is powerful! Let's learn."

10. Build a regex-based preprocessing Python function that:

- 1. Removes URLs**
- 2. Removes punctuation**
- 3. Tokenizes text**

```
print(preprocess("Learn NLP at https://nlp.ai! It's awesome."))
```

Mini Project Exercise

Regular Expressions for NLP with Python

Instructions

- Use Python and the **re** module only.
- Write clean, well-commented code.
- Do not use NLP libraries (NLTK, spaCy, etc.).
- Assume the input text is given.

Hello!!! I am studying NLP in 2026 😊.

Contact me at student123@gmail.com or admin@ucspyay.edu.mm.

Visit <https://nlp-course.org> for details.

The exam will be held on 12-03-2026.

NLP is soooo coooool!!!

Questions

1. Using `re.search()`, write a Python statement to check whether the given text contains any URL. Print "URL found" if a URL exists.
2. Using `re.match()`, check whether the given text starts with the word "Hello". Print an appropriate message.
3. Using `re.findall()`, extract all email addresses from the given text and store them in a list.
4. Using `re.finditer()`, extract the date in DD-MM-YYYY format and print:
 - the date
 - starting index
 - ending index

Questions

5. Using `re.sub()`, remove all URLs from the given text.
6. Using regex:
 - Remove emojis
 - Remove punctuation
 - Remove digits
 - Display the cleaned text.

Questions

7. Using `re.sub()`, normalize repeated characters so that no character appears more than two times consecutively.
8. Using `re.split()`, split the text into individual sentences.
9. Using `re.compile()`, tokenize the cleaned text into words and convert them to lowercase.

Questions

10. Write a Python **function `regex_nlp_pipeline(text)` that performs the following steps in order:**

- **Remove URLs**
- **Remove punctuation and emojis**
- **Remove numbers**
- **Normalize repeated characters**
- **Tokenize the text**

Return the final list of tokens.

Finite State Automata

https://colab.research.google.com/drive/12qu8rNLzC7GzT5nh_sayqMMONsyE4iC1?usp=sharing

FSA in Python

Language Pattern:

Determiner + Noun + Verb

Vocabulary:

DET = { the, a }

N = { dog, cat }

V = { barks, runs }

```
class SimpleFSA:
    def __init__(self):
        self.start_state = "q0"
        self.final_states = {"q3"}

        # Transition table
        self.transitions = {
            "q0": {"the": "q1", "a": "q1"},
            "q1": {"dog": "q2", "cat": "q2"},
            "q2": {"barks": "q3", "runs": "q3"}
        }
```

FSA in Python

```
def accept(self, sentence):
    words = sentence.lower().split()
    current_state = self.start_state

    for word in words:
        if current_state in self.transitions and word in self.transitions[current_state]:
            current_state = self.transitions[current_state][word]
        else:
            return False

    return current_state in self.final_states
```

FSA in Python

```
# Testing
fsa = SimpleFSA()

sentences = [
    "the dog barks",
    "a cat runs",
    "dog barks",
    "the cat"
]

for s in sentences:
    print(f"{s} -> {'Accepted' if fsa.accept(s) else 'Rejected'}")
```

Minimum Edit Distance

https://colab.research.google.com/drive/1QwRyr9xchG0Dsj7PG28pNPl_8D6tByfW?usp=sharing

N-gram Language Model

<https://colab.research.google.com/drive/1b6PgTh6CNbcDXwjHyhHvInf01OBqPdBO?usp=sharing>

HMM Language Model

<https://colab.research.google.com/drive/11vNCUrIMj6hDM4gLP12WcQIHG5rlaoT8?usp=sharing>

References

1. Natural Language Processing with Python, Steven Bird, Ewan Klein & Edward Loper, O'Reilly, 2009.
2. Corpus Linguistics, 2nd Edition, Tony McEnery & Andrew Wilson, 2001.
3. Speech and Language Processing_2025, 3rd Draft, Daniel Jurafsky & James H. Martin, 2025.
4. Foundations of Statistical Natural Language Processing, Christopher D. Manning & Hinrich Schutze, 1999.
5. Text Analytics with Python, Dipanjan Sarkar, 2016.
6. <https://www.nltk.org/book/ch02.html>
7. <https://github.com/nltk/nltk>